

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы № 3

по дисциплине «Программная инженерия»

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2026

Тема: формирование среды разработки ПС.

Цель: освоение технологии конфигурационного управления программным процессом, развертывания рабочей среды для разработки ПС, настройки и конфигурирования рабочей среды, создание репозитория для контроля версий ПС.

Задание:

1. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, провести обоснованный выбор средства управления данными и средства разработки моделей данных.
2. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, провести обоснованный выбор средств разработки специального ПО.
3. Создать удаленный репозиторий git (например, на github.com) с названием по имени проекта или по фамилии студента, создать инициализирующий коммит. При необходимости добавить к удаленному репозиторию участников проекта.
4. Провести идентификацию инструментальных и организационных рисков проекта.

Ход выполнения работы:

1. Выбор средства управления данными и разработки моделей данных

В соответствии с техническим заданием, разработанным в лабораторной работе №1, разрабатываемая ИПС предназначена для автоматизации процесса решения математических задач численными методами. Система должна накапливать историю решений для формирования обучающей выборки.

1.1. Выбор модели данных

Анализ требований к данным ПС показал, что сохраняемая информация (условие задачи, тип задачи, выбранный метод, параметры, результат) имеет нестрогую, слабо структурированную форму. Объекты данных могут содержать разный набор полей в зависимости от класса задачи (уравнение, интеграл и т.д.). Это делает нецелесообразным использование строгой реляционной модели с фиксированной схемой, так как это потребовало бы создания множества таблиц со сложными связями или использования механизмов наследования, что избыточно для данной задачи. Наиболее подходящей является документо-ориентированная (*NoSQL*) модель данных. Она позволяет хранить данные в виде JSON-документов, что соответствует

разнородной структуре решаемых задач и упрощает разработку, так как объекты JavaScript могут напрямую сохраняться в базу данных без преобразований.

1.2. Выбор системы управления базами данных (СУБД)

В соответствии с заданием проведен сравнительный анализ двух современных реляционных СУБД серверного типа — PostgreSQL и MySQL. Обе системы являются открытыми и широко используемыми в веб-разработке.

Критерий	PostgreSQL	MySQL
Тип лицензии	Бесплатная (BSD)	Бесплатная (GPL)
Операционные системы	Windows, Linux, macOS	Windows, Linux, macOS
Поддержка JSON	Есть (тип JSONB с индексацией)	Есть (тип JSON)
Поддержка транзакций	Полная (ACID)	Полная (ACID) в InnoDB
Производительность	Выше на сложных запросах	Выше на простых запросах
Расширяемость	Высокая (пользовательские функции)	Ограниченная
Интеграция с Python	asyncpg, SQLAlchemy	aiomysql, SQLAlchemy
Инструменты администрирования	pgAdmin, DBeaver	MySQL Workbench, phpMyAdmin
Документация	Подробная, много примеров	Подробная, много примеров

Таблица 1.1. Сравнительная характеристика серверных СУБД

Выбрала PostgreSQL, потому что:

- 1) У него лучше поддержка JSON (тип JSONB), а в проекте нужно хранить параметры задач, которые могут быть разными для уравнений, интегралов, ОДУ и систем
- 2) Он хорошо работает со сложными запросами, которые понадобятся для анализа истории решений и обучения нейросети
- 3) Легко подключается к FastAPI через библиотеку asyncpg
- 4) Полностью бесплатный и с открытым кодом
- 5) Есть удобный инструмент pgAdmin для управления базой данных

1.3. Выбор средства разработки модели данных

Для проектирования логической и физической модели данных выбрано CASE-средство ERwin по следующим причинам:

- 1) Поддержка реляционного моделирования "сущность-связь"
- 2) Возможность генерации SQL-скриптов для PostgreSQL

- 3) Обратное проектирование из существующей БД
- 4) Наглядное представление связей между таблицами

Альтернативно может использоваться встроенный инструмент pgAdmin для работы с PostgreSQL, однако ERwin обеспечивает более профессиональный уровень документирования модели.

2. Выбор средств разработки специального ПО

В соответствии с техническим заданием проведен анализ и выбор программных средств для разработки интеллектуальной программной системы

2.1. Выбор языка программирования

Для разработки серверной части системы рассматривались два современных языка программирования: Python и Node.js (JavaScript).

Критерий	Python	Node.js (JavaScript)
Типизация	Динамическая (есть type hints)	Динамическая (есть TypeScript)
Производительность	Средняя, достаточно для API	Высокая, асинхронная
Экосистема для ML	TensorFlow, PyTorch, scikit-learn	TensorFlow.js (ограниченная)
Асинхронность	Поддерживается (async/await)	Встроенная
Веб-фреймворки	FastAPI, Django, Flask	Express, NestJS
Работа с БД	SQLAlchemy, asyncpg	Sequelize, TypeORM
Математические библиотеки	NumPy, SciPy	Math.js (базовые операции)
Скорость разработки	Высокая	Высокая

Таблица 2.1. Сравнение языков программирования

Выбран Python, так как он обеспечивает полноценную поддержку TensorFlow для обучения нейросетей, имеет богатые математические библиотеки для реализации численных методов и отличается простотой разработки, что важно для учебного проекта

2.2. Выбор среды разработки

Для разработки проекта необходимо выбрать интегрированную среду разработки (IDE), которая обеспечит комфортную работу с языками Python и JavaScript, а также интеграцию с системой контроля версий Git

Критерий	Visual Studio Code	PyCharm
Тип лицензии	Бесплатная	Бесплатная (Community Edition)

Поддержка Python	Отличная (плагины)	Встроенная
Поддержка JavaScript	Отличная	Хорошая
Терминал	Встроенный	Встроенный
Интеграция с Git	Встроенная, удобная	Встроенная
Потребление оперативной памяти	Немного	Больше
Плагины и расширения	Огромное количество	Много

Таблица 2.2 — Сравнение сред разработки

Выбрана Visual Studio Code, так как проект включает и серверную часть на Python, и клиентскую на JavaScript. VS Code позволяет работать с обоими языками в одной среде, потребляет меньше ресурсов и имеет удобную интеграцию с Git. PyCharm более тяжелый и ориентирован только на Python

2.3. Выбор платформы разработки и сопровождения ПО

В качестве платформы для хостинга кода и совместной работы рассматривались GitHub и GitLab

Критерий	GitHub	GitLab
Тип лицензии	Бесплатно для публичных репозиторий	Бесплатно
Встроенный CI/CD	GitHub Actions	GitLab CI/CD
Система отслеживания задач	GitHub Issues	GitLab Issues
Интеграция с внешними сервисами	Широкая	Широкая
Популярность	Очень высокая	Высокая

Таблица 2.3. Сравнение платформ разработки и сопровождения ПО

Выбран GitHub, поскольку репозиторий уже создан на этой платформе, она проста в использовании и имеет большое количество документации и примеров для учебных проектов.

2.4. Выбор средства управления жизненным циклом ПО (CI/CD)

Для автоматизации сборки, тестирования и развертывания приложения рассматривались GitHub Actions и Jenkins

Критерий	GitHub Actions	Jenkins
Размещение	Облачное (встроено в GitHub)	Требуется собственный сервер
Настройка	Через YAML в репозитории	Через интерфейс или Jenkinsfile
Интеграция с GitHub	Встроена	Требуется настройка

Бесплатность	Бесплатно для публичных репозиторий	Бесплатно (open source)
Сложность настройки	Низкая	Высокая

Таблица 2.4. Сравнение средств управления жизненным циклом ПО

Выбран GitHub Actions, поскольку он не требует отдельного сервера, интегрирован с GitHub и прост в настройке, что оптимально для учебного проекта. Jenkins потребовал бы дополнительных ресурсов и времени на администрирование

3. Создание репозитория проекта

В соответствии с заданием для версионного контроля разрабатываемой системы создан удаленный репозиторий на платформе GitHub

Ссылка на репозиторий: <https://github.com/Vikkkaa123/intelligent-math-solver>

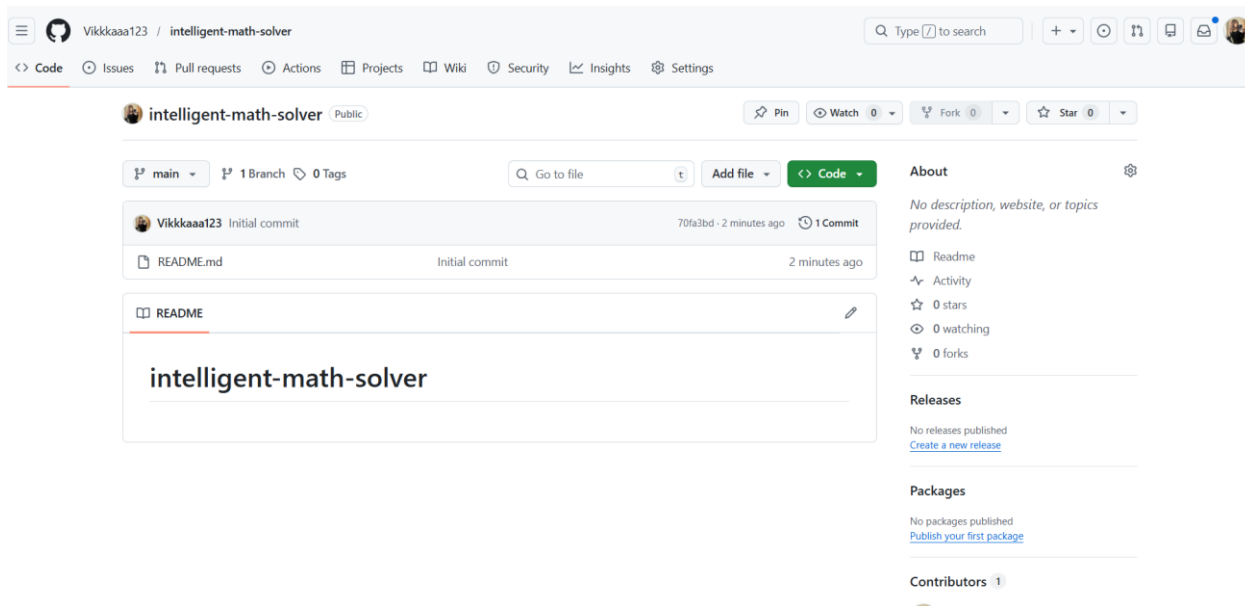


Рисунок 1. Главная страница репозитория

При создании репозитория был автоматически сгенерирован инициализирующий коммит с файлом README.md

4. Идентификация рисков проекта

На основе анализа проектных решений (лабораторные работы №1-2) и выбранных инструментальных средств (лабораторная работа №3) идентифицированы следующие риски:

4.1. Инструментально-технологические риски

Шифр	Риск (что может произойти)	Последствия
T-1	Обнаружение несовместимости версий TensorFlow и FastAPI на этапе интеграции	Невозможность обучить нейросеть, потребуется переустановка библиотек, срыв сроков
T-2	Потеря соединения с БД из-за неверных параметров в строке подключения	Ошибки при сохранении результатов, потеря данных текущей сессии
T-3	Блокировка CORS на сервере, не настроенная для клиентской части	Фронтенд не сможет отправлять запросы, система неработоспособна
T-4	Сбой жесткого диска или случайное удаление папки с проектом	Полная потеря кода и накопленной истории решений
T-5	Переполнение оперативной памяти при обработке большого количества точек в Chart.js	Зависание или аварийное завершение работы браузера
T-6	Некорректная обработка ошибок при парсинге математических выражений	Непредсказуемое поведение, некорректные результаты вычислений

Таблица 4.1 — Инструментально-технологические риски

4.2. Организационные риски

Шифр	Риск	Последствия риска
O-1	Невозможность собрать достаточную тестовую выборку для обучения нейросети вручную	Низкое качество распознавания запросов, необходимость синтезировать данные
O-2	Отсутствие доступа к интернету в аудитории для установки зависимостей	Невозможность продемонстрировать работу системы на защите
O-3	Недостаточное комментирование кода и отсутствие пояснений к алгоритмам	Трудности при объяснении работы методов на защите проекта
O-4	Неправильное распределение времени между изучением новых технологий и написанием кода	Риск не уложиться в сроки сдачи работы

Таблица 4.2 — Организационные риски

Идентифицированные риски будут учитываться при дальнейшей разработке. Для минимизации рисков планируется регулярное резервное копирование, тестирование на разных этапах и четкое планирование времени

Вывод: в ходе выполнения лабораторной работы №3 была сформирована среда разработки интеллектуальной программной системы для решения математических задач. В соответствии с техническим заданием проведен обоснованный выбор средств управления данными и средств разработки специального ПО. Создан удаленный репозиторий на GitHub, выполнена инициализация проекта с первым коммитом. Проведена идентификация инструментально-технологических и организационных рисков, предложены меры по их минимизации